



A Proposal for a New Incentive Mechanism on Cortex (Bittensor Subnet 100): Bounty and Proof

Verified research as the unit of work: operator-signed topics, a digest-pinned autonomous judge, sealed baselines, and network-scale learning from proven artifacts

Cortex Foundation

Technical report · Version 1.1 · September 2026

Abstract. Cortex is Bittensor Subnet 100: decentralised, collaborative AI research organised as challenges whose outcomes are scored and turned into on-chain incentive. Two challenges are live, *Bounty* and *Proof*, and this report proposes them as a replacement for the dominant subnet pattern in which a miner ships a harness that a static, observable evaluator scores—a pattern that pays for the generalisation gap rather than for capability, spends compute on non-transferable weights, hides data and recipe choices, and defeats audit. *Proof* makes the reproducible experiment the unit of work. Miners publish a claim, a code artifact and a declared compute budget against operator-signed, dynamically injected topics spanning pre-training, post-training, agent harnesses, systems, optimisers and data. A digest-pinned Recursive Language Model agent judges reproduction, cheating and contamination; a deterministic harness measures private holdouts the judge never sees against baselines sealed before the topic opened; topics settle winner-take-all or in a discovery mode whose pass floor pays for reproducible research and whose remainder pays for improvement and novelty on a dominance lattice. *Bounty* is a bug-hunting challenge: independent hunters report defects across `cortex.foundation` and Cortex applications (product surfaces and services) so the operator can keep continuous production service with the fewest defects for clients. Adjudication is human-judged and served from public read-only endpoints that validators consume and anyone can replay. Both fail closed. Despite centralised evaluation, every paid quantity is publicly verifiable through digests, signatures, commitments, sealed baselines and open artifacts, and paying for proven findings rather than private checkpoints lets a digest-pinned synthesiser compound them into a shared, self-improving stack. The figures carry the argument; the text states the claims and the formal core.

1 Introduction

Bittensor pays miners from a block emission according to weights that validators set and a stake-weighted consensus reconciles [10, 12]. What a subnet produces is therefore decided by its scoring procedure. Cortex, Subnet 100, does not serve a single model or task; it organises research as challenges, each a self-contained scoring module whose normalised scores validators compose into their weight vector (Figure 1). The two live challenges divide the space of work: *Proof* for claims a machine can verify, *Bounty* for production defects that need human judgement. The allocation of emission between them is an operator-tuned parameter that this report deliberately does not state.

The report makes four claims. Scoring a shipped harness with a static observable evaluator is structurally suboptimal for a research network (Section 2). A market for *verified research*, in which an autonomous, digest-pinned judge reproduces claims and a blind harness measures them against sealed baselines, removes the incentive to overfit and makes recipes public by construction (Sections 3 and 4). Centralised evaluation is compatible with public verifiability when every crossing of the trust boundary is bound by a hash, a signature or a commitment (Sections 5 and 6). And paying for proven findings rather than private checkpoints lets knowledge compound into a shared stack that isolated training can never build (Section 7).

2 Why shipping a harness fails

Let $\mathcal{H}$ be a space of shippable objects (weights, pipelines, prompt programs), $\mathcal{D}$ the distribution the network cares about, and $u(h, x) \in [0, 1]$ a utility. A static evaluator holds one sample D^{vis} of size n and reports $\hat{S}(h)$; the network wants $S^*(h)$:

$$S^*(h) = \mathbb{E}_{x \sim \mathcal{D}}[u(h, x)], \quad \hat{S}(h) = \frac{1}{n} \sum_{x \in D^{\text{vis}}} u(h, x), \quad \Gamma(h) = \hat{S}(h) - S^*(h). \quad (1)$$

Because payment is monotone in $\hat{S}$, a unit of Γ is worth as much to the miner as a unit of S^* and is cheaper to buy. With T adaptive submissions a miner can drive $\Gamma = \Theta(\sqrt{T/n})$ while leaving S^* unchanged [1, 4]; leaderboards in the wild show the effect [13, 14], and for language models it reappears as contamination [11, 15]. Four consequences follow (Figure 2): gains accrue to the gap, compute is spent on non-transferable weights, recipe and data are invisible to the evaluator, and an auditor who replays $\hat{S}$ learns nothing about S^* . A fifth is operational: when the evaluator fails, naive clients pay a uniform or stale default, a *paid zero*. A replacement must score on data the miner cannot query against a target it cannot move, score an object that carries its recipe, publish enough to recompute every payment, and abstain on failure.

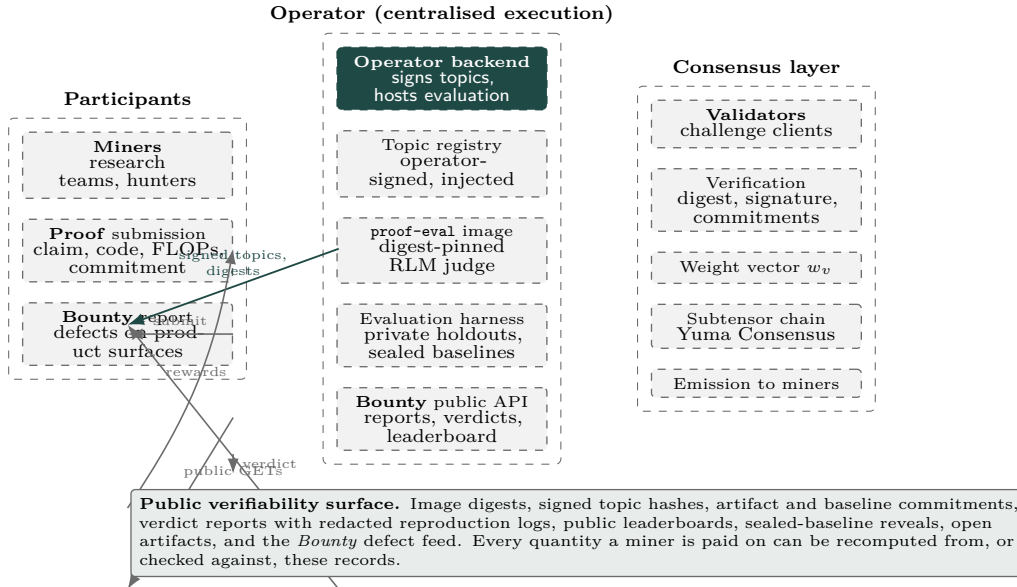


Figure 1: Architecture of Cortex (Subnet 100). Miners submit experiments to *Proof* and defect reports to *Bounty*. The operator executes evaluation centrally: it signs and injects topics, hosts the digest-pinned **proof-eval** judge image, runs the deterministic harness with private holdouts and sealed baselines, and serves the *Bounty* public API over reports filed against `cortex.foundation` and Cortex applications. Validators do not evaluate; they verify the image digest, topic signatures and commitments, read scores and public GET endpoints, compose the two challenge vectors into a weight vector w_v , and Yuma Consensus turns weights into emission. Validators need no GPUs; their job shifts from computation to verification.

3 Proof: verified research as the unit of work

A *Proof* topic τ is a signed record: a research question, a metric set K_τ with orientations and scales, a **payout_mode** that is **wta** or **discovery**, a topic weight ω_τ , a submission window, commitments to private holdouts and to a sealed baseline b_τ , and an English **validation** block with three fields: **score_on** (what the harness measures and how submissions rank), **accept_if** (what a reproduced claim must satisfy) and **reject_if** (conditions that end evaluation regardless of the metric: network access at evaluation time, overlap with a committed holdout, excluded data provenance, hard-coded outputs). Topics are injected dynamically by the operator and signed on entry, so miners cannot pre-optimize against targets not yet posed, and their scope is unbounded: agent harnesses, post-training recipes, pre-training data mixtures, decentralised training without high-bandwidth interconnects [3, 6], optimisers, tokenisers, evaluation methods. Figure 3 gives the lifecycle.

A *submission* a is a claim κ , an open code artifact A at a pinned revision with an environment and data manifest, a declared compute F , and a commitment $\text{Com}(A)$ published before judging. The artifact is the recipe: disclosure is the price of payment. The judge is a Recursive Language Model agent [16] shipped as the **proof-eval** image with pinned digest d . It plans its own investigation, reproduces the claim in a network-isolated sandbox at declared or reduced scale, checks for cheating and contamination [2, 5], and emits a categorical verdict; it never sees holdout metrics, which the deterministic harness computes behind an information barrier together with off-path canary checks and the comparison to the sealed baseline (Figure 4). The image contains no simulation switch, the digest being public evidence of that absence, and fail-closed guards yield HTTP 503 and abstention when the digest is empty, no topic is open, or a baseline is unsealed.

Formal core of *Proof* scoring. Orient and normalise metrics so larger is better; $m(a) \in \mathbb{R}^K$ is the harness vector, b_τ the sealed baseline, $\alpha \in \mathbb{R}_{>0}^K$ topic weights with $\sum_k \alpha_k = 1$. With the pass gate $g(a) = \mathbf{1}[\nu(a) = \text{pass}]$, the frontier before a is the componentwise join $F_\tau^-(a) = b_\tau \vee \bigvee_{a' \prec a, g(a')=1} m(a')$, and

$$\delta(a) = \sum_{k=1}^K \alpha_k [m_k(a) - F_{\tau,k}^-(a)]_+, \quad S_\tau(a) = g(a) [\beta + (1 - \beta)q(a)], \quad q(a) = \lambda \hat{\delta}(a) + (1 - \lambda)\eta(a), \quad (2)$$

where $\hat{\delta} = \min(1, \delta/\delta_\tau^{\max})$, $\eta(a) \in [0, 1]$ is the distance of the judge’s method descriptor from earlier accepted methods, β is the pass floor and λ weights improvement against novelty. Under **discovery** the topic payout is $p_\tau(a) = S_\tau(a) / \sum_{a'} S_\tau(a')$ over accepted submissions; under **wta** it is $\mathbf{1}[a = \arg \max_{a'} (\|m(a') \vee b_\tau - b_\tau\|_{\alpha,1} - t(a'))]$, earliest commitment breaking ties. A miner’s *Proof* score is $s_i^P \propto \sum_\tau \omega_\tau \sum_{a \in \mathcal{A}_i} p_\tau(a)$, undefined ($\perp$) under any fail-closed guard, and validators compose $w_v = \phi(s^B, s^P; \rho)$ with ρ an operator-tuned allocation not published here; ϕ renormalises over defined vectors and skips the update if none is defined.

Proposition 1 (Telescoping and nullity). *For accepted $a_1, \dots, a_n$ in commitment order, $\sum_j \delta(a_j) = \|F_\tau - b_\tau\|_{\alpha,1}$; a submission dominated by the frontier before it has $\delta = 0$. Hence resubmission, splitting an advance across submissions, and Sybil hotkeys cannot inflate the improvement a topic pays.*

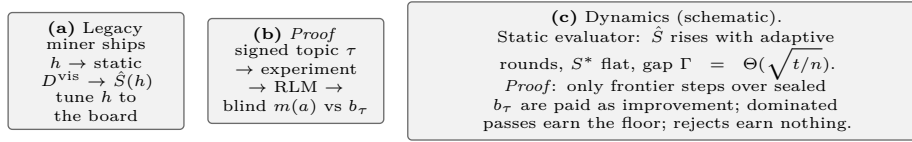


Figure 2: Legacy harness market versus *Proof*. (a) A miner ships a fixed object h to a static evaluator on an observable sample and tunes h to the leaderboard; the optimised quantity is Γ , not S^* . (b) In *Proof* the object is a reproducible experiment against a dynamically injected, operator-signed topic; only verified deltas are paid and enter the public corpus. (c) Axes without units: under a static evaluator the visible score rises while the true score is flat; under *Proof* only steps by which the frontier $F_\tau(t)$ rises above b_τ are paid as improvement.

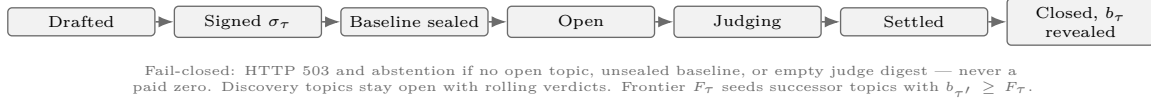


Figure 3: Lifecycle of a *Proof* topic. A topic is drafted, signed under the published operator key ($\sigma_\tau = \text{Sign}_O(H(\tau))$), and cannot open until $\text{Com}(b_\tau)$ is published. The baseline is measured by the same harness on the same holdouts that will score submissions. A **wt**a topic settles once; a **discovery** topic remains open. On closure the baseline is revealed with its salt so anyone can check it against the commitment and against every report.

Proposition 2 (Zero value of overfitting). $S_\tau(a)$ depends on a only through g and η , functions of the artifact under a pinned judge, and through $m(a)$, computed on holdout data independent of anything the miner can query; effort aimed at the visible record therefore has zero expected return.

4 Payout modes

The scoring rule in (2) separates three questions (Figure 5): whether a result is real, which the gate answers; whether reproducible research should be paid even when it does not move the frontier, which the pass floor β answers positively, because a corpus needs negative and marginal results and reproducibility has a cost; and how the remainder should be split, which the lattice increment and novelty answer by paying for the frontier advance a submission causes and for the distance of its method from earlier accepted methods. **wt**a topics suit questions where only reaching a target first matters, such as matching a stated synchronous-training loss with a decentralised optimiser at a stated bandwidth. **discovery** topics suit open questions where diverse partial answers have value. Declared compute F is checked, not paid: a declaration inconsistent with the observed reproduction is evidence of a **cheat**, and the operator calibrates β so that the expected floor on a typical topic is of the order of the compute a reproducible experiment of that kind requires. Descriptor-based deduplication and a per-topic cap on floors keep the floor from being farmed.

5 Bounty: bug-hunting on production surfaces

Some defects cannot be scored by a machine looking at a sealed holdout: a broken product flow on **cortex.foundation**, a failing Cortex application surface, an authentication or billing edge, a gateway that 500s on a well-formed request. *Bounty* is the challenge that pays independent hunters to find those defects and report them, so the operator can keep continuous production service with the fewest defects for clients (Figure 6).

A hunter pairs a Bittensor hotkey to a dedicated Cortex account and files a report against a product surface or service: title, body, and reproduction steps. The operator (or an operator-authorized adjudicator) tags each report with a verdict

$$\nu_B \in \{\text{valid}, \text{already_fixed_not_prod}, \text{invalid_malicious}, \text{duplicate}\} \quad (3)$$

and, when $\nu_B = \text{valid}$, a severity $z \in \{\text{trivial}, \text{minor}, \text{major}, \text{critical}\}$ that prices impact. Unpriced valid rows are not creditable: an unpriced bug cannot be paid for. The scarce resource is adjudication, not a holdout. The three volume plays and their answers are precision, impact, and a triage-noise canary kept off the paid number:

$$\text{precision}_i = \frac{V_i}{V_i + M_i}, \quad \text{impact}_i = \bar{z}_i, \quad r_i = \text{precision}_i \cdot \text{impact}_i, \quad (4)$$

where V_i and M_i are the hunter's valid and malicious counts and $\bar{z}_i$ is the mean assigned severity. Junk subtracts. Forty cosmetic findings at perfect precision are worth a fraction of four critical ones. Duplicates and already-fixed re-files earn nothing and raise a triage-noise ratio that is reported but not in the lattice; above a published cap they are a hard zero. Champion is displacement versus the previous bounty champion on r_i , subject to a precision floor so beating a sloppy incumbent is not enough.

The backend exposes the published reports, justifications, severities and the current leaderboard through read-only GET endpoints that require no credentials. Validators consume exactly these endpoints and normalise over the active set,

$$s_i^B = r_i / \sum_j r_j \quad (5)$$

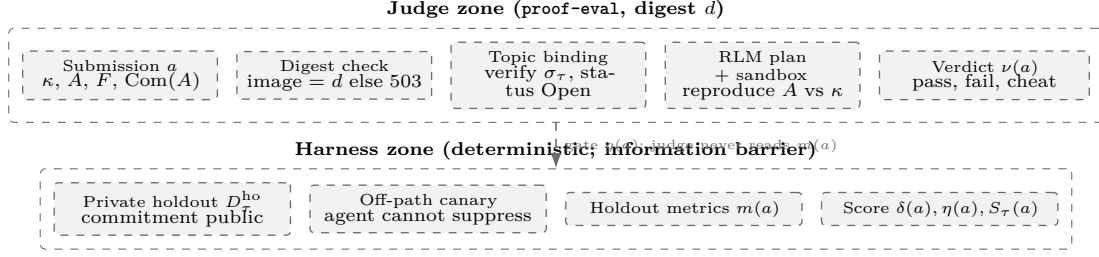


Figure 4: The digest-pinned RLM judge and the blind harness. Intake verifies the running image equals d and that the topic is authentic and open. The agent decomposes the claim, reproduces the artifact, and analyses cheating and contamination, then emits $\nu(a)$. The harness fills $m(a)$ and the sealed-baseline comparison. The information barrier means the judge never reads $m(a)$, so a verdict cannot be rationalised from a number and the public transcript cannot leak the holdout.

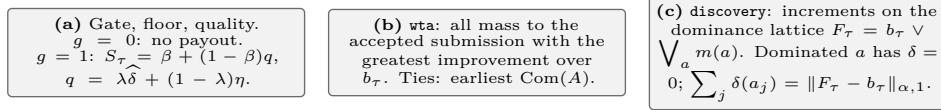


Figure 5: Payout modes, conceptually. (a) The verdict gates payment; a passing submission’s score is a pass floor for research cost plus a quality that combines normalised lattice increment and novelty. (b) Under **wta** the whole topic weight goes to the accepted submission with the greatest improvement over the sealed baseline. (c) Under **discovery** improvement is the α -weighted advance of the componentwise join. Increments telescope to the total frontier advance, so the improvement mass a topic pays is fixed by what the topic actually achieves. Novelty $\eta(a)$ is the distance of the method descriptor from earlier accepted methods, so a second route to the same outcome is still rewarded while a copy is not.

when the feed is readable; any third party can replay the same requests and must obtain the same vector as the weights on chain. If the backend URL is unset or unreachable the challenge client abstains: no synthetic scores, no replay of a cached allocation, no uniform default. Human judgement decides whether a report is a real defect on a live surface, but its record is complete and machine-readable, so a subjective decision becomes an auditable one, and a hunter cannot inflate payment by filing more junk or by splitting one finding.

6 Public verifiability despite centralised evaluation

Four things run in one place: the judge image, topic authoring, the live holdout contents and the pre-reveal baseline. Each is bound to a public record by a primitive whose violation would be detectable (Figure 8). The judge has a public identity, its content digest d [9]: any change to code, prompts, tools or harness changes d , and validators reject verdicts from any other image. Topics are signed under the operator key, so no one else can inject one and the evaluator cannot score against an unpublished one. Holdouts and baselines are committed before use and opened on schedule, when anyone can recompute a report’s metrics by running the revealed holdout through the pinned harness on the open artifact; artifacts are committed before judging, fixing what was judged and establishing priority; every verdict has a public report. *Bounty* publishes the full adjudication trail. The residual trust is narrow and explicit: that the live holdout equals the committed one (checked at rotation), that the operator does not leak holdouts (statistically detectable as divergence between live and post-reveal performance of a favoured miner, and permanent in the record), and that topics are authored in good faith (a governance question with a complete signed history). A static local evaluator offers transparency of its live input instead, which by Section 2 is exactly what makes it overfittable.

7 Network-scale learning from proven research

The mechanism serves a thesis: Cortex should learn at network scale rather than from fragmented private training runs. Consider N miners each training a checkpoint with compute C under a quality law $Q(C) = Q_\infty - AC^{-\gamma}$ [8] with realised qualities $q_i = Q(C) + \varepsilon_i$. The network can adopt one checkpoint, so its usable value is $V_{\text{iso}}(N) = \mathbb{E} \max_i q_i \simeq Q(C) + \sigma\sqrt{2 \ln N}$: flat in N up to a selection bonus, while $(N - 1)C$ of compute is discarded and the recipes stay private, so nothing carries to the next round. Under *Proof* the network’s knowledge is not a maximum but a join: the frontier $F_\tau = b_\tau \vee \bigvee_a m(a)$ together with the open artifact realising each coordinate, and $\bigvee_a m(a) \geq m(a^*)$ for every single artifact, strictly whenever two findings improve different metrics. Verification costs a production fraction $r < 1$ of each accepted run, so the compute without network-level return is rNC rather than $(N - 1)C$.

The intended end state (Figure 7) is a second digest-pinned agent, the synthesiser $\Sigma_{d'}$, which reads the corpus of proven artifacts once and proposes an update θ'_t to the shared stack: harness, training recipe, data policy. The proposal is not adopted on its own authority; it is submitted to the same pipeline against a topic whose sealed baseline is the current stack, and $\theta_{t+1} = \theta'_t$ only if it passes and dominates, so the shared stack is monotone in every metric and every adoption has a public report. The synthesiser learns from proven research, never from raw miner weights. The loop reduces contamination, since every finding entered through the gates and canaries with a public data manifest; it keeps dangerous or unvetted datasets out of the shared stack, since the provenance gate rejects excluded sources and the data policy changes only through the verified route; and it still pushes the capability frontier, since topics are unbounded and the synthesiser adopts whatever passes sealed evaluation. What is excluded is not ambition but routes to a high score



Figure 6: Bounty adjudication and its public verification. Independent hunters report defects across `cortex.foundation` and Cortex applications (product surfaces and services). The operator records a verdict and, for a creditable valid, a severity. The backend serves every artefact of that judgement from a public read-only API. A validator polls the endpoints, forms s^B from precision times impact, and contributes it to w_v . Anyone can replay the same GETs. The client fails closed when the backend is not configured or not reachable.

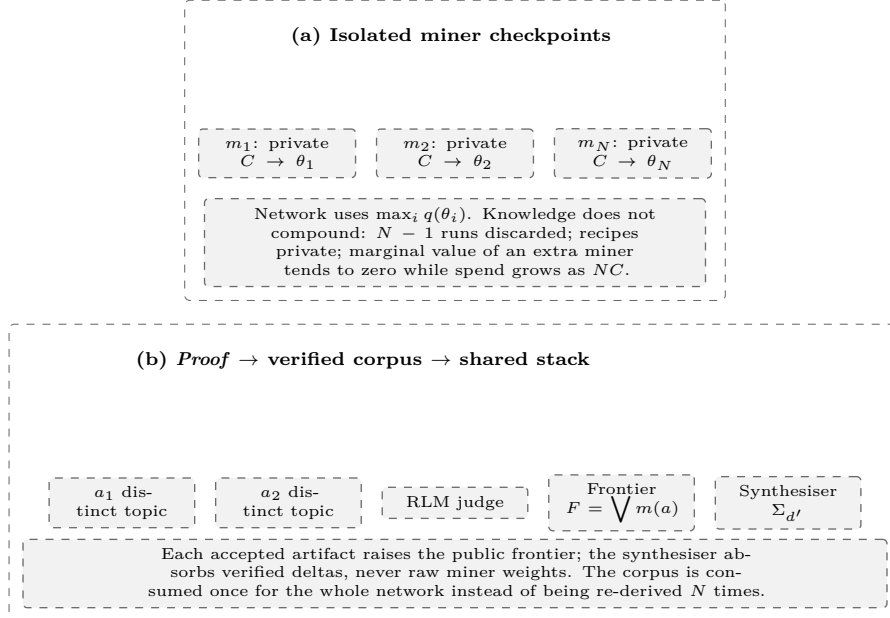


Figure 7: Isolated miner checkpoints versus a verified research corpus that self-improves the shared stack. (a) N miners each run a private training job of compute C ; the network can use only $\max_i q(\theta_i)$. (b) Under *Proof*, miners run distinct experiments on distinct topics; accepted artifacts raise a public frontier $F = \bigvee_a m(a)$, a join that no single artifact need have achieved. A digest-pinned synthesiser reads the corpus once and proposes a recipe update that becomes the new shared stack only if verified.

that do not reproduce or transfer.

8 Conclusion

A research subnet must pay for what research produces: findings that are true, reproduce, are attributable and transfer. The harness pattern pays for the generalisation gap, hides the recipe and defeats audit; *Proof* scores reproducible experiments on data the miner cannot query against baselines it cannot move, with a judge that is autonomous yet pinned and a lattice rule under which duplication cannot inflate payment; *Bounty* pays independent hunters to report defects across `cortex.foundation` and Cortex applications so continuous production service stays low-defect for clients, and makes that human adjudication replayable; both abstain rather than pay a default. Centralised execution and public verifiability coexist because every crossing of the trust boundary is hashed, signed or committed and opened on schedule. The limitations are stated: operator trust during a topic's live window is detectable rather than preventable, the judge can err and its error rate should be published per digest, reproduction bounds the scale at which claims are checked directly, English validation blocks trade expressiveness for interpretive variance, and the synthesiser is the intended end state rather than a deployed component. Future work moves topic authorship to threshold signing, evaluation toward trusted execution or succinct proofs [7], and validators toward sampled re-judging after holdout rotation.

The mechanism is live in its two challenges; the public record it produces is the evidence on which it should be judged.

References

- [1] A. Blum and M. Hardt. The ladder: A reliable leaderboard for machine learning competitions. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 1006–1014, 2015.
- [2] N. Carlini, C. Liu, Ú. Erlingsson, J. Kos, and D. Song. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX Security Symposium*, pages 267–284, 2019.
- [3] A. Douillard, Q. Feng, A. A. Rusu, R. Chhaparia, Y. Donchev, A. Kuncoro, M. Ranzato, A. Szlam, and J. Shen. DiLoCo: Distributed low-communication training of language models, 2023. arXiv:2311.08105.

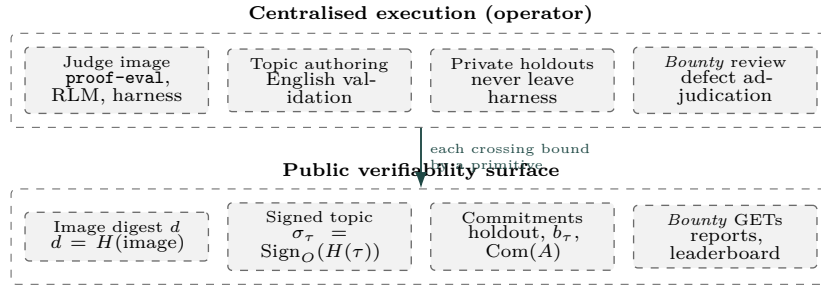


Figure 8: The trust boundary. Left, centralised execution: evaluation code and the judge image, topic authoring, private holdouts, baseline measurement, full judge transcripts and *Bounty* review. Right, the public verifiability surface. Each crossing is bound by a primitive: content addressing for the judge, a signature for topics, commit-now-open-later for holdouts, commit-before-open-reveal-at-close for baselines, publication with holdout values redacted for transcripts, and full publication for defect adjudication. The trade is about *when* a quantity can be checked, not *whether*.

- [4] C. Dwork, V. Feldman, M. Hardt, T. Pitassi, O. Reingold, and A. Roth. The reusable holdout: Preserving validity in adaptive data analysis. *Science*, 349(6248):636–638, 2015.
- [5] S. Golchin and M. Surdeanu. Time travel in LLMs: Tracing data contamination in large language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [6] S. Jaghouar et al. INTELLECT-1 technical report, 2024. arXiv:2412.01152.
- [7] D. Kang, T. Hashimoto, I. Stoica, and Y. Sun. Scaling up trustless DNN inference with zero-knowledge proofs, 2022. arXiv:2210.08674.
- [8] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models, 2020. arXiv:2001.08361.
- [9] Open Container Initiative. OCI image format specification. <https://github.com/opencontainers/image-spec>, 2017.
- [10] Opentensor Foundation. Yuma consensus. Bittensor documentation, <https://docs.bittensor.com/yuma-consensus>, 2024.
- [11] Y. Oren, N. Meister, N. S. Chatterji, F. Ladhak, and T. B. Hashimoto. Proving test set contamination in black-box language models. In *International Conference on Learning Representations (ICLR)*, 2024.
- [12] Y. Rao, J. Steeves, A. Shaabana, D. Attevelt, and M. McAteer. Bittensor: A peer-to-peer intelligence market. Technical report, Opentensor Foundation, 2021. <https://bittensor.com/whitepaper>.
- [13] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar. Do ImageNet classifiers generalize to ImageNet? In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, pages 5389–5400, 2019.
- [14] R. Roelofs, V. Shankar, B. Recht, S. Fridovich-Keil, M. Hardt, J. Miller, and L. Schmidt. A meta-analysis of overfitting in machine learning. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.
- [15] O. Sainz, J. A. Campos, I. García-Ferrero, J. Etxaniz, O. Lopez de Lacalle, and E. Agirre. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of EMNLP 2023*, pages 10776–10787, 2023.
- [16] A. L. Zhang, T. Kraska, and O. Khattab. Recursive language models, 2025. arXiv:2512.24601.